¿Qué es XSD?

XSD (XML Schema Definition) es un lenguaje para **definir la estructura y las reglas de validación** de un documento XML. Es el método moderno para validar XML y supera ampliamente las limitaciones de DTD.

Elementos y atributos

Qué elementos existen y qué atributos tienen

Tipos de datos

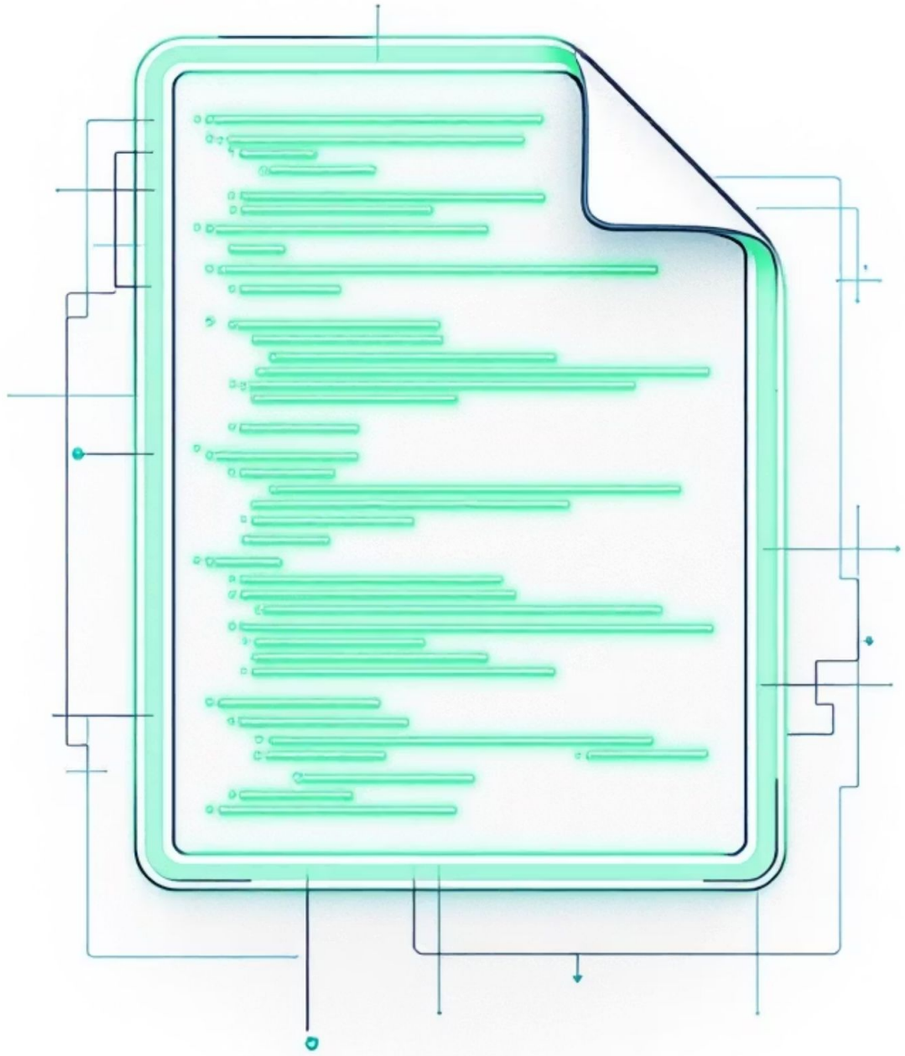
Qué tipo de dato debe contener cada campo

Ocurrencias

Cuántas veces puede aparecer un elemento

Restricciones

Qué reglas y límites deben cumplirse



Objetivo y características de XSD

¿Para qué sirve?

XSD garantiza la **integridad y consistencia** de los datos en XML. Verifica que la estructura sea correcta, que los datos tengan el tipo adecuado y que se cumplan las reglas definidas.

```
<persona>
  <nombre>Ana</nombre>
  <edad>25</edad>
</persona>
```

XSD valida que `nombre` sea texto y `edad` sea un entero.

Ventajas frente a DTD

- Escrito en **XML** nativo
- Soporte de **tipos de datos** predefinidos
- **Restricciones de valores** avanzadas
- **Reutilización** de tipos y estructuras
- Soporte de **namespaces**

Todo esquema XSD comienza con: `<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">` — donde `xs` es el prefijo del espacio de nombres.

Ejemplo básico de validación

XML a validar

```
<persona>
  <nombre>Ana</nombre>
  <edad>25</edad>
</persona>
```

Esquema XSD correspondiente

```
<xs:schema xmlns:xs="...W3C...">
 <xs:element name="persona">
 <xs:complexType>
 <xs:sequence>
 <xs:element name="nombre"
 type="xs:string"/>
 <xs:element name="edad"
 type="xs:integer"/>
 </xs:sequence>
 </xs:complexType>
 </xs:element>
</xs:schema>
```

Tipos de datos en XSD

Una de las grandes ventajas de XSD frente a DTD es la disponibilidad de **tipos de datos predefinidos** que permiten una validación precisa y robusta.

Tipo	Descripción
`xs:string`	Texto libre
`xs:integer`	Número entero
`xs:decimal`	Número decimal
`xs:boolean`	Verdadero o falso
`xs:date`	Fecha (AAAA-MM-DD)
`xs:time`	Hora (HH:MM:SS)

Ejemplo de uso

```
<xs:element name="precio"
  type="xs:decimal"/>
```

XML válido:

```
<precio>19.99</precio>
```

Usar el tipo correcto evita errores silenciosos: XSD rechazará `"veinte"` donde se espera un `xs:decimal`.

Elementos simples y complejos

Elemento simple

Contiene **texto o un valor**. Es la unidad mínima de datos en un esquema XSD.

```
<xs:element name="nombre"
  type="xs:string"/>
```

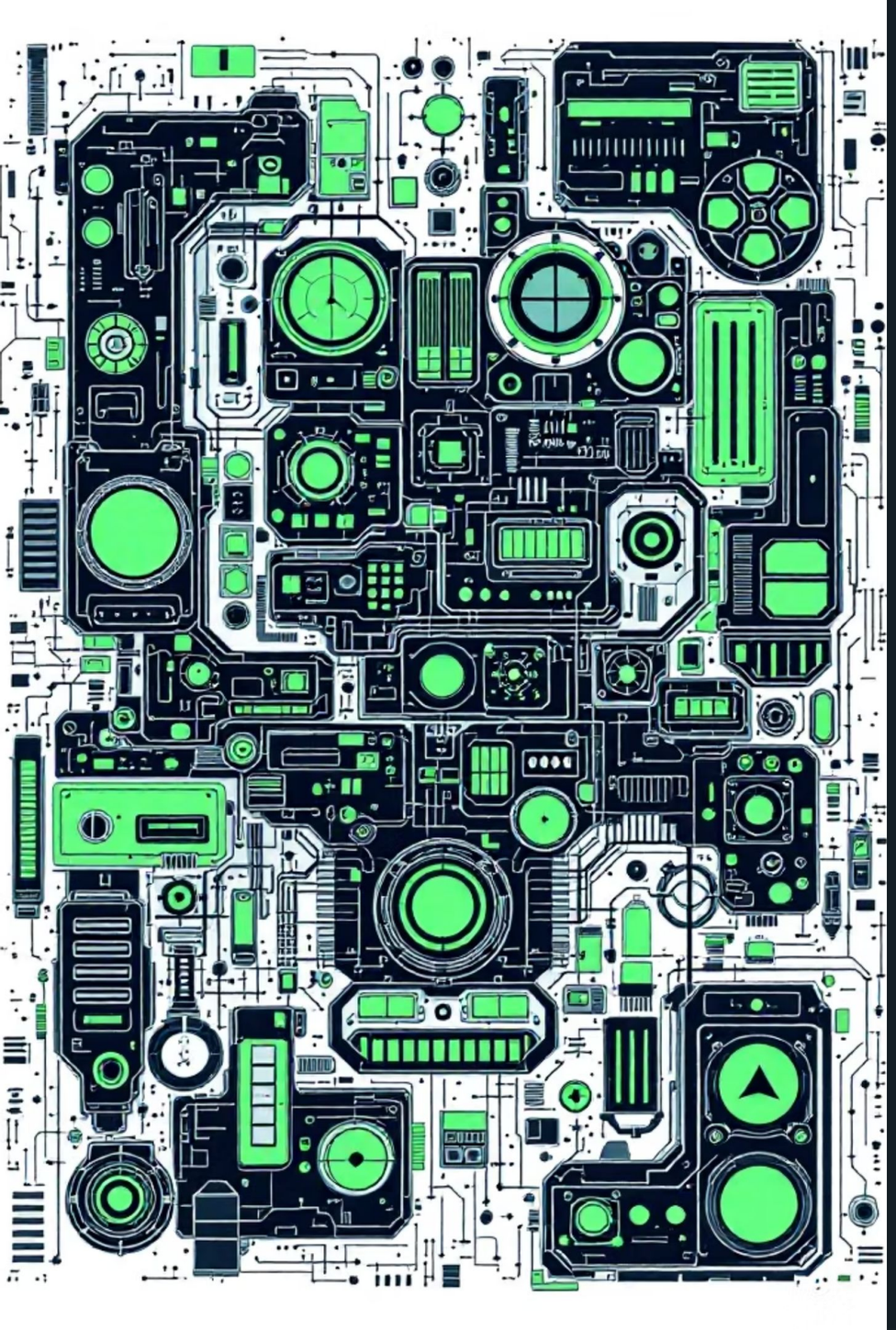
XML válido:

```
<nombre>Ana</nombre>
```

Elemento complejo

Contiene **subelementos o atributos**, permitiendo definir estructuras jerárquicas anidadas.

```
<xs:complexType>
  <xs:sequence>
    <xs:element name="nombre"
      type="xs:string"/>
    <xs:element name="edad"
      type="xs:integer"/>
  </xs:sequence>
</xs:complexType>
```



Control de ocurrencias

XSD permite especificar cuántas veces puede aparecer un elemento mediante los atributos `minOccurs` y `maxOccurs`.

Definición en XSD

```
<xs:element name="telefono"
  type="xs:string"
  minOccurs="0"
  maxOccurs="3"/>
```

minOccurs="0"

El elemento es **opcional**

maxOccurs="3"

Puede aparecer hasta **3 veces**

XML válido resultante

```
<contacto>
  <telefono>123456</telefono>
  <telefono>987654</telefono>
</contacto>
```

Usar `maxOccurs="unbounded"` permite repeticiones ilimitadas del elemento.

Tipos de estructura: sequence, choice y all

sequence — Orden obligatorio

Los elementos deben aparecer **en el orden exacto** definido en el esquema. Ideal para estructuras de datos estrictas.

```
<xs:sequence>
  <xs:element name="nombre"/>
  <xs:element name="edad"/>
</xs:sequence>
```

choice — Elemento alternativo

Solo puede aparecer **uno de los elementos** definidos. Útil para modelar opciones mutuamente excluyentes.

```
<xs:choice>
  <xs:element
name="telefono"/>
  <xs:element name="email"/>
</xs:choice>
```

all — Orden libre

Todos los elementos deben aparecer **exactamente una vez**, pero en cualquier orden.

```
<xs:all>
 <xs:element name="nombre"/>
 <xs:element name="edad"/>
</xs:all>
```

Atributos en XSD

Definición y valores de use

Los atributos se declaran dentro de un `complexType`, tras los elementos hijos.

Valor	Significado
`optional`	El atributo es opcional
`required`	El atributo es obligatorio
`prohibited`	El atributo no está permitido

Ejemplo completo

```
<xs:element name="libro">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="titulo"
        type="xs:string"/>
      <xs:element name="autor"
        type="xs:string"/>
    </xs:sequence>
    <xs:attribute name="isbn"
      type="xs:string"
      use="required"/>
  </xs:complexType>
</xs:element>
```

XML válido:

```
<libro isbn="978-84-376-0494-7">
 <titulo>1984</titulo>
 <autor>George Orwell</autor>
</libro>
```

Restricciones con facets

XSD permite definir **restricciones precisas sobre los valores** mediante **facets** aplicados con `xs:restriction`.

Longitud mínima

```
<xs:restriction
  base="xs:string">
  <xs:minLength
    value="3"/>
</xs:restriction>
```

El texto debe tener al menos 3 caracteres.

Rango numérico

```
<xs:restriction
  base="xs:integer">
  <xs:minInclusive
    value="0"/>
  <xs:maxInclusive
    value="120"/>
</xs:restriction>
```

Útil para validar una edad entre 0 y 120.

Valores permitidos

```
<xs:restriction
  base="xs:string">
  <xs:enumeration
    value="admin"/>
  <xs:enumeration
    value="usuario"/>
</xs:restriction>
```

Solo se aceptan los valores enumerados.

Resumen: XSD en la práctica

XSD es la herramienta estándar para garantizar la **validez estructural y semántica** de documentos XML en entornos profesionales.

Define la estructura

Declara elementos simples y complejos con sus tipos de datos.

Controla ocurrencias

Usa `minOccurs` y `maxOccurs` para flexibilidad.

Organiza con sequence, choice o all

Elige el modelo de contenido según el caso de uso.

Añade restricciones con facets

Garantiza la integridad de los datos con reglas precisas.